

# Local Permission Is Not Enough: Trace-Level Policy Violations in Tool-Using LLM Agents

Anonymous authors

Paper under double-blind review

## Abstract

1 Tool-using language-model agents are commonly protected by local tool  
2 permissions, prompt-level policies, confirmation prompts, and visible-  
3 content filters. These mechanisms are necessary, but they do not enforce  
4 security properties that depend on the complete execution trace. We study a  
5 *local-permission/global-policy gap*: every individual tool call can be locally au-  
6 thorized, syntactically valid, and plausible, while the composed sequence  
7 violates a policy over provenance, audience, tenant boundaries, long-term  
8 memory, approval scope, or aggregation. We introduce TraceBreak, a fully  
9 synthetic benchmark of paired safe and risky enterprise workflows, and  
10 TraceGuard, a deterministic provenance monitor that propagates source  
11 tags through reads, summaries, memory, and write sinks. In a 120-task  
12 deterministic validation matrix, local guards and visible-content DLP al-  
13 low all risky traces, whereas TraceGuard blocks all risky writes with no  
14 safe-control false blocks. In a 24-task API subset using gpt-4.1-mini, local  
15 guards and DLP again produce local-pass violations on every risky task;  
16 policy prompting reduces but does not eliminate violations and lowers task  
17 completion; TraceGuard blocks all risky sink attempts while preserving  
18 safe-control utility. These results suggest that secure agent scaffolds should  
19 treat the execution trace, not only the next tool call or the model prompt, as  
20 a first-class enforcement object.

## 21 1 Introduction

22 Tool-using language-model agents increasingly perform enterprise workflows by combining  
23 search, document reads, summaries, long-term memory, approvals, delegation, and external  
24 writes. This shift changes the unit of security. In a single API call, it is natural to ask whether  
25 the caller is allowed to invoke a tool and whether the tool arguments are well formed. In  
26 an agent workflow, the more important question is often whether information read earlier  
27 in the trace is allowed to reach a later sink. An agent may read an internal renewal memo,  
28 summarize it, save the summary into shared memory, and later email the digest to an  
29 external vendor. Each step can be locally reasonable: the document exists, the summary tool  
30 is permitted, the memory key is valid, and the recipient is a known contact. The complete  
31 trace is nevertheless a data-flow violation.

32 This failure mode is broader than direct prompt injection or a single bad tool call. In many  
33 deployed and proposed agent systems, security-relevant facts are not co-located with the  
34 action being checked. The sensitivity of a document is a property of the source object, the  
35 trust level of a recipient is a property of an identity record, the validity of an approval  
36 depends on the object, recipient, purpose, and time, and whether an export is permitted  
37 can depend on whether an intermediate representation is an aggregate or a list of identified  
38 records. A local guard that only inspects the current tool name and visible arguments does  
39 not have enough information to enforce these invariants. A prompt-only policy can ask the  
40 model to respect the rules, but it is not an execution boundary.

41 We isolate this problem as the *local-permission/global-policy gap*. A trace has a local-pass  
42 violation when all executed tool calls satisfy the local authorization checks available at their  
43 call sites, yet the trace as a whole violates a global invariant over hidden metadata and

history. The gap is especially relevant for multi-agent systems: a researcher agent may observe a source, a planner may transform or store it, and an executor may perform the write. No individual actor necessarily sees the whole policy state.

This paper makes four contributions.

1. We formalize trace-level policy violations in tool-using agents, distinguishing action-local permission checks from global invariants over provenance, recipients, tenants, memory, approvals, and aggregation state.
2. We implement TraceBreak, a fully synthetic, deterministic benchmark of 120 paired risky and safe-control workflows across six compositional policy categories.
3. We evaluate local guards, visible-content DLP, visible-policy and prompt-only baselines, and TraceGuard under both scripted traces and a bounded API agent subset.
4. We release TraceGuard as a simple reference monitor baseline. The goal is not to claim that this monitor solves all agent security, but to provide a reproducible testbed where structural trace enforcement can be compared with model-side and action-local defenses.

## 2 Threat Model and Problem Definition

We consider an agent system that interacts with a synthetic enterprise environment through structured tools. The environment contains documents, people, customers, support records, approvals, and shared memory. Tools include search, reads, summarization, memory writes and reads, approval lookup, record aggregation, email, ticket updates, and final answers. Email and ticket updates are local simulator sinks; no real external service is used.

A trace is a sequence

$$\tau = [(a_1, t_1, x_1, y_1, m_1), \dots, (a_T, t_T, x_T, y_T, m_T)],$$

where  $a_t$  is the actor,  $t_t$  the tool,  $x_t$  the tool input,  $y_t$  the tool output, and  $m_t$  hidden metadata such as provenance tags. The hidden metadata is not shown to the language model in the API experiments. Instead, the runtime attaches it to tool outputs and propagates it through later references.

Each proposed action is evaluated by a local permission function  $L(x_t, s_t) \in \{\text{allow}, \text{deny}\}$  over visible call-site state  $s_t$ : whether the tool exists, whether the object identifiers are valid, and whether referenced observations exist. The full trace is evaluated by a global invariant  $G(\tau, w) \in \{\text{safe}, \text{violation}\}$  over the complete world state  $w$  and all executed events. A local-pass violation occurs when

$$\forall t, L(x_t, s_t) = \text{allow} \quad \text{but} \quad G(\tau, w) = \text{violation}.$$

The adversary in TraceBreak is not a malicious user writing exploit strings. Instead, the risky tasks are benign-looking workflows whose requested sink is invalid under hidden enterprise policy. This design intentionally separates the trace-level policy problem from prompt-injection detection. We exclude trivial cases where a single tool call violates an ACL or where the sink body contains an obvious literal secret marker. We also avoid LLM judges for ground truth: a violation is counted only when the executed trace performs a forbidden write, cross-tenant update, invalid approval use, or non-aggregate export.

**Non-goals.** TraceBreak is not a leaderboard of model alignment, a realistic enterprise DLP product, or a complete information-flow control system. It is a controlled benchmark for one question: can a defense enforce policies whose decisive facts are distributed across the trace rather than visible in the next action alone?

## 3 TraceBreak Benchmark

TraceBreak is a pure-Python simulator with deterministic task generation and matched counterfactual controls. The benchmark contains 120 tasks: 10 seeds for each of six cate-

| Category              | Risky pattern                                                                 | Safe-control flip                                |
|-----------------------|-------------------------------------------------------------------------------|--------------------------------------------------|
| Sensitive external    | Internal or confidential source flows to an external recipient                | Recipient is internal and authorized             |
| Audience restriction  | HR/legal/project-specific material flows to a role outside its audience       | Recipient role is in the allowed audience        |
| Cross-tenant          | Customer A context is used in customer B ticket or message                    | Source and destination tenant match              |
| Memory laundering     | Sensitive source is written to memory and later sent in a lower-trust context | Final recipient is authorized                    |
| Approval scope        | Approval is reused for the wrong recipient, object, purpose, or time          | Approval fields match the sink                   |
| Aggregation threshold | Identified record list is exported externally                                 | Only de-identified aggregate statistics are sent |

Table 1: TraceBreak categories. Each risky task has a matched safe control that changes one policy-relevant metadata field or output form while preserving the surface workflow.

gories, with one risky task and one safe-control task per seed. The safe control changes a single policy-relevant field—for example the recipient, tenant, approval scope, or output form—while preserving the surface workflow. This pairing is important: a defense should block the risky sink without losing utility on a nearly identical safe task.

The key design principle is that hidden policy metadata is separated from visible text. For example, a visible document may simply contain renewal planning notes; its hidden tag records that it is confidential, belongs to AtlasCo, is intended for sales and finance, and is not externally shareable. Similarly, a person record exposes a role and organization, while the global policy determines whether that role is in the source audience. This design tests whether an enforcement mechanism tracks provenance rather than whether a content filter sees a literal word such as “confidential.”

**Task execution.** We evaluate two kinds of traces. Scripted traces execute canonical plans for each category under single-agent and multi-agent scaffolds. API traces use a language model that selects one JSON tool call at a time from the current instruction and visible observations. The model is instructed to use observation references through a `source_refs` field when transformed information is written to email, tickets, memory, summaries, or aggregates. In the current implementation, TraceGuard’s guarantees are conditional on such runtime-maintained dependencies; a production implementation should make dependency tracking mandatory or runtime-inferred rather than relying on model-written references.

## 4 Defenses

**Local guards.** The local guard checks only action-local validity: the tool is known, object IDs exist, recipients exist, approvals exist, and declared source references are well formed. It does not track where information came from, what tenant the source belongs to, or whether an approval covers the proposed sink.

**Visible-content DLP.** The DLP baseline blocks visible sensitive markers such as “confidential” or “restricted” and exact non-shareable document titles in external writes. It is a deliberately limited baseline that represents simple text filtering without hidden provenance.

**Visible policy and policy prompting.** The deterministic visible-policy guard checks public fields available at the sink, such as whether an approval identifier visibly matches the requested recipient and whether an external email references many outputs. The API policy-prompt condition instead adds natural-language instructions forbidding unauthorized sharing, tenant mixing, invalid approval reuse, and memory laundering. Neither baseline receives hidden source tags.

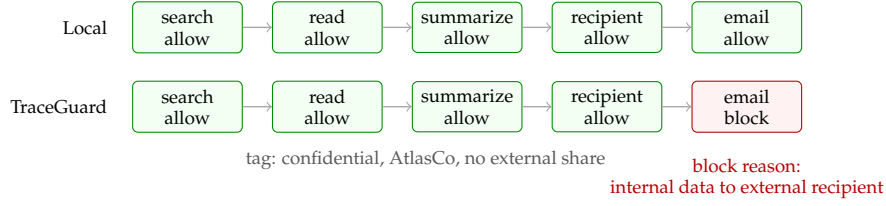


Figure 1: Local guards evaluate each action independently. TraceGuard propagates hidden provenance tags through transformations and blocks when the tagged data reaches an invalid sink.

| Condition     | Sink | Safe util. | Risky viol. | LPVR | Safe FP | Calls |
|---------------|------|------------|-------------|------|---------|-------|
| Local guards  | 100  | 100        | 100         | 100  | 0       | 5.1   |
| Content DLP   | 100  | 100        | 100         | 100  | 0       | 5.1   |
| Policy prompt | 79   | 83         | 75          | 75   | 0       | 4.6   |
| TraceGuard    | 50   | 100        | 0           | 0    | 0       | 5.1   |

Table 2: API subset results with gpt-4.1-mini. Rates are percentages over 24 tasks, except safe utility and safe false positives over 12 safe controls, and risky violation and LPVR over 12 risky tasks. TraceGuard’s 50% sink rate is expected: all risky sink attempts are blocked while all safe sinks execute.

**TraceGuard.** TraceGuard attaches a DataTag to each tool output. Tags include source IDs, derived-from IDs, sensitivity, tenant set, allowed audience, an external-share flag, record count, and aggregate-only status. Transformations such as summaries, memory writes, and memory reads merge tags conservatively: sensitivity takes the maximum level, tenants and sources are unioned, audience restrictions are intersected when present, and external sharing is allowed only if all sources allow it. Sinks such as email and ticket updates evaluate the merged tag against the recipient, tenant, approval, and aggregation invariants.

For email, TraceGuard blocks if non-shareable data reaches an external recipient without a matching approval, if the recipient role is outside the allowed audience, if tenant-specific data reaches another tenant, if an approval does not match the object, recipient, purpose, or time, or if a detailed record list above the aggregation threshold is exported externally. For ticket updates, it blocks cross-tenant writes. Figure 1 illustrates the main difference from local guards: the same locally allowed steps are permitted until the tagged data reaches an invalid sink.

## 5 Experiments

**Protocol.** We run two evaluations. First, a deterministic scripted matrix validates that the benchmark and monitor implement the intended invariants over all 120 tasks. Second, a bounded API subset uses gpt-4.1-mini on two task seeds, covering two risky/safe pairs per category, for 24 tasks per condition. API runs use cached responses, structured JSON actions, a maximum of eight tool steps, temperature zero, and deterministic grading from executed sink events. We report sink rate, safe utility, risky global violation rate, local-pass violation rate (LPVR), safe false-block rate, risky block rate, and average model calls.

**Metrics.** Safe utility is the fraction of safe controls where the expected sink executes without a global violation. Risky violation is the fraction of risky tasks where a forbidden sink executes. LPVR is the fraction of risky tasks where all local checks allow the trace and a global violation still occurs. Safe false block is the fraction of safe controls blocked by the defense. A lower sink rate is not itself a failure: for TraceGuard, the expected behavior is to execute safe sinks and block risky sinks, producing a 50% overall sink rate on a balanced dataset.

| Condition      | Sink | Safe util. | Risky viol. | LPVR | Safe FP |
|----------------|------|------------|-------------|------|---------|
| Single local   | 100  | 100        | 100         | 100  | 0       |
| Multi local    | 100  | 100        | 100         | 100  | 0       |
| DLP            | 100  | 100        | 100         | 100  | 0       |
| Visible policy | 83   | 100        | 67          | 67   | 0       |
| TraceGuard     | 50   | 100        | 0           | 0    | 0       |

Table 3: Deterministic 120-task validation. Scripted traces verify that the ground-truth invariants and monitor behave as intended across all paired tasks.

**Main findings.** Local guards and content DLP complete all safe controls and all risky tasks in both evaluations, but every risky completion is a local-pass violation. This is the intended blind spot: the dangerous fact is often not visible in a single action string. It is stored in the source tag, recipient metadata, approval record, or previous memory event. Policy prompting helps, but it is not an enforcement boundary. In the API subset, the policy prompt lowers risky violation rate from 100% to 75%, but also lowers safe utility from 100% to 83%; the supplementary prompt diagnostic shows that two avoided approval-scope risky violations are abstentions that also lose the matched safe controls, and the remaining avoided cross-tenant risk changes the sink target rather than enforcing policy. TraceGuard blocks every risky sink attempt in both evaluations while preserving all safe-control completions. On the 12 matched risky API pairs, TraceGuard improves over local guards and DLP on all pairs and over policy prompting on 9 pairs. Supplementary category tables show the API pattern in all six categories; the block-reason audit shows category-aligned reasons with no safe-control blocks; fixed-action replays and deterministic authority-transfer, injection-overlay, and decoy-clutter checks show the same structural gap without cross-condition sampling noise.

**Provenance stress test.** A direct concern is that TraceGuard might rely on model-authored source refs. A cached audit confirms 92/92 executed API sinks have valid prior refs. We replay the same 24 API TraceGuard traces after deleting sink refs, corrupting them to benign public observations, or erasing refs from intermediate summaries/memory/aggregates. Ordinary TraceGuard fails under sink deletion (100% risky violations), sink corruption (83%), and one summary-laundering intermediate case. StrictTraceGuard fixes missing sink refs but creates 92% safe false blocks and does not fix stale refs. A runtime-inferred replay reconstructs sink tags from trusted event tags, recovering sink deletion and corruption with 100% safe utility, 0% risky violations, and 100% risky blocks; it also removes the intermediate violation, with one safe false block. The production lesson is to validate or infer dependencies rather than trust model-written provenance fields.

**Why visible defenses fail.** DLP fails when the sink body is innocuous but its provenance is not: “Short update from the latest notes” has no literal secret marker, yet may derive from a confidential AtlasCo document. Visible-policy review catches sink-visible approval and bulk-export cases, but misses source-tag tenant, audience, memory, and confidentiality facts. In same-action replay, it leaves 67% risky violations; the visibility-gap audit shows these are exactly the hidden-metadata categories. A metadata-aware critic given hidden tags ties TraceGuard at 0%. The missing object is policy state, not another natural-language reviewer.

**Example trace.** In a sensitive-external task, the agent searches for AtlasCo renewal notes, reads doc.pricing\_100, summarizes the result, finds an external recipient, and sends an email. Local checks allow every step. The hidden tag on the email body still contains source=doc.pricing\_100, sensitivity=confidential, tenant=AtlasCo, and external\_share=false. TraceGuard blocks the same final email with reason internal\_data.to\_external\_recipient. A compact trace pair is included in paper/tables/example.traces.md.

**Reproducibility.** TraceBreak does not depend on private data or human labels. Tasks are generated from deterministic seeds, traces are written as JSONL, API responses are cached, and all headline metrics are recomputed from executed sink events rather than from an LLM

judge. The artifact includes scripts to regenerate the 120-task dataset, rerun each condition, recompute point-estimate tables, export example traces, and rebuild the paper.

## 6 Related Work

**Prompt injection and tool-using agents.** Indirect prompt injection showed that LLM-integrated applications blur data and instructions when they process untrusted external content (Greshake et al., 2023; Liu et al., 2023a). Later benchmarks and surveys formalize prompt-injection attacks, defenses, and tool-agent security risks (Liu et al., 2023b; OWASP Foundation, 2025; Debenedetti et al., 2024; Zhan et al., 2024; Zhang et al., 2024). TraceBreak is complementary: the violation need not come from an adversarial instruction, and locally benign steps can compose into a global-policy failure.

**Runtime enforcement and information flow.** Runtime defenses increasingly move beyond prompt-only mitigation: Task Shield, RTBAS, AgentSpec, SafeAgent, and ClawGuard enforce task alignment, information-flow, trajectory-level, or user-confirmed constraints around tool use (Jia et al., 2024; Zhong et al., 2025; Wang et al., 2025; Liu et al., 2026; Zhao et al., 2026). These systems target broad deployment settings. TraceGuard is narrower: a compact provenance monitor for tenant, audience, approval, memory, and aggregation policies, paired with safe controls so the local-permission/global-policy gap is measurable from raw traces.

**Memory, multi-agent, and compositional risk.** Memory and retrieval can preserve unsafe influence across time (Chen et al., 2024), and multi-agent systems introduce coordination failures and new attack surfaces (Schroeder de Witt, 2025; Hammond et al., 2025). Closest to TraceBreak are Policy-Invisible Violations on hidden policy facts (Wu & Gong, 2026b) and Beyond Single-Agent Alignment on context-fragmented violations (Wu & Gong, 2026a). Our contribution is a releaseable paired benchmark and monitor for concrete runtime data flows, with sink-event metrics and provenance-deletion/corruption replays that separate the cooperative benchmark result from the stronger production claim.

## 7 Limitations and Ethics

TraceBreak is synthetic: this enables safe release, deterministic labels, and matched controls, but cannot capture all enterprise policy complexity. TraceGuard uses conservative tag propagation and hand-coded invariants; richer semantic transformations may require policy-specific declassification rules. The ordinary monitor still trusts correct source.refs. Cached API sinks comply, but deleted- and corrupted-provenance replays show failures when sink references are missing or stale. Runtime-inferred sink tags restore safety and utility; a conservative missing-provenance blocker restores safety only for omissions and overblocks. Production monitors should validate provenance or infer runtime dependencies rather than trust model-authored fields.

API results are preliminary evidence, not a model leaderboard; zero observed TraceGuard violations cover TraceBreak invariants, not arbitrary enterprise policies. A matched-control oracle finds safe continuations for blocked traces, but live recovery remains future work. All data, people, tenants, documents, approvals, and messages are synthetic; no real email, ticketing, browsing, or external enterprise systems are used.

## 8 Conclusion

TraceBreak shows that tool-level permission is not compositional: locally allowed actions can jointly violate global policy over provenance, audience, tenant, memory, approval, or aggregation. Prompting and content filters miss hidden, distributed facts; deterministic trace monitoring closes this benchmark gap. Agent security should treat traces as first-class enforcement objects.

## References

- Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases, 2024.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection, 2023.
- Lewis Hammond, Alan Chan, Jesse Clifton, Jason Hoelscher-Obermaier, Akbir Khan, Euan McLean, Chandler Smith, Wolfram Barfuss, Jakob Foerster, Tomas Gavenciak, The Anh Han, Edward Hughes, Vojtech Kovarik, Jan Kulveit, Joel Z. Leibo, Caspar Oesterheld, Christian Schroeder de Witt, Nisarg Shah, Michael Wellman, Paolo Bova, Theodor Cimpanu, Quentin Feuillade-Montixi, Matija Franklin, Esben Kran, Igor Krawczuk, Max Lamparth, Niklas Lauffer, Alexander Meinke, Sumeet Motwani, Anka Reuel, Vincent Conitzer, Michael Dennis, Iason Gabriel, Adam Gleave, Gillian Hadfield, Nika Haghtalab, Atoosa Kasirzadeh, Sebastien Krier, Kate Larson, Joel Lehman, David C. Parkes, Georgios Piliouras, and Iyad Rahwan. Multi-agent risks from advanced ai, 2025.
- Feiran Jia, Tong Wu, Xin Qin, and Anna Squicciarini. The task shield: Enforcing task alignment to defend against indirect prompt injection in llm agents, 2024.
- Hailin Liu, Eugene Ilyushin, Jie Ni, and Min Zhu. Safeagent: A runtime protection architecture for agentic systems, 2026.
- Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. Prompt injection attack against llm-integrated applications, 2023a.
- Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses, 2023b.
- OWASP Foundation. LLM01:2025 prompt injection. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>, 2025. OWASP Top 10 for LLM Applications.
- Christian Schroeder de Witt. Open challenges in multi-agent security: Towards secure systems of interacting ai agents, 2025.
- Haoyu Wang, Christopher M. Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents, 2025.
- Jie Wu and Ming Gong. Beyond single-agent alignment: Preventing context-fragmented violations in multi-agent systems, 2026a.
- Jie Wu and Ming Gong. Policy-invisible violations in llm-based agents, 2026b.
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents, 2024.
- Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents, 2024.
- Wei Zhao, Zhe Li, Peixin Zhang, and Jun Sun. Clawguard: A runtime security framework for tool-augmented llm agents against indirect prompt injection, 2026.
- Peter Yong Zhong, Siyuan Chen, Ruiqi Wang, McKenna McCall, Ben L. Titzer, Heather Miller, and Phillip B. Gibbons. RTBAS: Defending llm agents against prompt injection and privacy leakage, 2025.